Assign 3

과제 풀이 정리 · Notion 아카이브

UCS - frontier.update(state,cost) | frontier.removeMin()

```
class UniformCostSearch(SearchAlgorithm):
def __init__(self, verbose: int = 0):
    self.verbose = verbose
----------------------------------------------------------------
def solve(self, problem: Searchproblem, heuristic=lambda x:0)
self.actions = None
self.totalCost = None
slef.numStatesExplored = 0

# Initialize data structures
frontier = priorityQueue()
backpointers = {}

# Add the start state
startState = problem.startState()
frontier.update(startState, 0)

  While True:
  state, pastCost = frontier.removeMin()
  if state == None: break   //-> queue가 나오지 않았으면(비어있었음), 끝
  self.numStatesExplored += 1

# Check if we've reached an end state; if so, extract solution.
if problem.isEnd(state):
self.actions = []
while state != startState:            //-> 2중 반복문
                    action, prevState = backpointers[state]
                    self.actions.append(action)
                    state = prevState
self.actions.reverse()
self.totalCost = pastCost
return
```

```
# Expand from |state| to new successor states,
    # updating the frontier with each newState.
    for action, newState, cost in problem.succAndCost(state):
    if frontier.update(newState, pastCost+cost+heuristic(newState)) - heuristic(state)):
    backpointers[newState] = (action, state)

  // update 됐으면 -> backpointers[] 에 저장 (더 작은 값이 오면 -> 업데이트 됨)
  // backpointers : state -> action, prevState
```

Problem 1 : *Language Models and Search Cost 1*

1a÷Word Segmentation(단어 나누기)

띄어쓰기가 없는 string 입력받고

-> 그럴듯한 곳에 띄어쓰기를 해야하는 문제

```
==========================================================
class WordSegmentationProblem(util.SearchProblem):
def __init__(self, query:str, unigramCost: Callable[[str],float])
self.query = query
self.unigramCost = unigramCost
----------------------------------------------------------
def startState(self) -> int:
return 0
----------------------------------------------------------
def isEnd(self, state: int) -> bool:
return state == len(self.query)
----------------------------------------------------------
def succAndCost(self,state: int) -> iterable[Tuple[str, int, float]]:
succAndCost_list = []
for end_idx in range(state+1, len(self.query)+1)
succ = self.query[state:end_idx]     //-> 한 단어
cost = self.unigramCost(succ)        // 그 단어의 cost
succAndCost_list.append((succ,end_idx,cost))
return succAndCost_list
==========================================================
def segmentWords(query: str, unigramCost: Callable[[str],float])->str:
ucs = util.UniformCostSearch(verbose=0)
ucs.solve(WordSegmentationProblem(query, unigramCost))
```

return " ".join(ucs.actions)

1b÷ Exact-k Word Segmentation

띄어쓰기가 없는 string + k

-> k개의 단어로 만듦

==========

class ExactKWordSegmentationProblem(util.SearchProblem):

def __init__(self,query:str,k:int,unigramCost:Callable[[str],float])

self.query = query

self.k = k

self.unigramCost = unigramCost

---

def startState(self) -> Tuple[int,int]

return (0,0) <- 현재 idx , 단어 개수

---

def isEnd(self, state:Tuple[int,int]) -> bool:

return state[0] == len(self.query) and state[1] == self.k

---

def succAndCost(self, state: Tuple[int,int])->Iterable[Tuple[str,Tuple[int,int],float]]

if state[1] == self.k // 탐색 중에, 단어 개수 끝남

return []

succAndCost_list = []

for end_inx in range(state[0]+1,len(self.query)+1):

succ = self.query[state[0]:end_idx]

cost = self.unigramCost(succ)

succAndCost_list.append((succ,(end_idx,state[1]+1),cost))

return succAndCost_list

==========

def segmentExactKWords(query:str, k:int, unigramCost:Callable[[str],float])->str:

if len(quert) == 0 or len(query) < k or k == 0

return ""

ucs = util.UniformCostSearch(verbose=0)

ucs.solve(ExactWordSegmentationProblem((query,k,unigramCost))

if ucs.actions == None:

return ""

return " ".join(ucs.actions)

Problem 2 : *Language Models and Search Cost 2*

2a÷Vowel Insertion(모음 삽입)

```
class VowelInsertionProblem(util.SearchProblem):
  def __init__(
    self,
    queryWords: List[str],
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]],
  ):
  self.queryWords = queryWords
  self.bigramCost = bigramCost
  self.possibleFills = possibleFills
----------------------------------------------------------------
def startState(self) -> Tuple[int, str]:
return (0, wordsegUtil.SENTENCE_BEGIN)    <- state[1] = 전 단어
----------------------------------------------------------------
def isEnd(self, state: Tuple[int,str]) -> bool:
return state[0] == len(self.queryWords)
----------------------------------------------------------------
def succAndCost(self, state: Tuple[int,str]) -> Iterable[Tuple[str, Tuple[int,str],float]]:
if len(self.possibleFills(self.queryWords[state[0]])) == 0:
succ = queryWords[state[0]]
cost = bigramCost(queryWords[state[1]], succ)
return [(succ,(state[0]+1, succ),cost)]
succAndCost_list = []
for succ in self.possibleFills(self.queryWords[state[0]]):
cost = self.bigramCost(state[1], succ)
succAndCost_list.append((succ,(state[0]+1,succ),cost))
return succAndCost_list
================================================================
def insertVowels(
  queryWords: List[str],
  bigramCost: Callable[[str, str], float],
  possibleFills: Callable[[str], Set[str]],
) -> str:
if len(queryWords) == 0:
return ::
```

```
ucs = util.UniformCostSearch(verbose=0)
ucs.solve(VowelInsertionProblem(queryWords,bigramCost,possibleFills))
return " ".join(ucs.acitons)
```

2b÷ Restricted-Vowel Insertion `set(c).isdisjoint(restricted_set)`

```
restrictedVowels: str -> 금지어
---------------------------------------------------------------
class RestrictedVowelInsertionProblem(util.SearchProblem):
def __init__(
  self,
    queryWords: List[str],
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]],
    restrictedVowels: str,
  ):
    self.queryWords = queryWords
    self.bigramCost = bigramCost
    self.possibleFills = possibleFills
    self.restrictedVowels = restrictedVowels
---------------------------------------------------------------
def startState(self) -> Tuple[int, str]:
return (0, wordsegUtil.SENTENCE_BEGIN))
---------------------------------------------------------------
def isEND(self, state: Tuple[int,str])->bool:
return state[0] == len(queryWords)
---------------------------------------------------------------
def succAndCost(self, state: Tuple[int,str])->Iterable[Tuple[str, Tuple[int,str],float]]:
queryW = self.queryWords[state[0]]
candidates = self.possibleFills(queryW)
restricted_set = set(self.restrictedVowels) <- self.restrictedVowels:str이라서 set으로 변경
filtered_candidates = {
c for c in candidates                  // c는 단어 vs restricted_set은 한 글자
if set(c).isdisjoint(restricted_set)   // c가 restricted_set에 겹치는지
}
if len(filtered_candidates) == 0:
cost = self.bigramCost(state[1],QueryW)
return [(QurryW,(state[0]+1,QueryW),cost)]
succAndCost_list = []
```

for succ in sroted(filtered_candidates):

cost = self.bigramCost(state[1],succ)

succAndCost_list.append((succ,(state[0]+1,succ),cost))

return succAndCost_list

==========

def insertRestrictedVowels(

  queryWords: List[str],

  bigramCost: Callable[[str, str], float],

  possibleFills: Callable[[str], Set[str]],

  restrictedVowels: str,

) -> str:

if len(queryWords) == 0:

return ""

ucs = util.UniformCostSearch(verbose=0)

ucs.solve(RestrictedVowelInsertionProblem(queryWords,bigramCost,possibleFills,restrictedVowels))

return " ".join(ucs.actions)

Problem 3 : *Putting It Together*

3a : Putting It Together (Joint Segmentation + Insertion)

띄어쓰기, 모음 모드 제거된 자음만 남은 문자열 -> 문으로 복구

---

class JointSegmentationInsertionProblem(util.SearchProblem):

  def __init__(

    self,

    query: str,

    bigramCost: Callable[[str, str], float],

    possibleFills: Callable[[str], Set[str]],

  ):

    self.query = query

    self.bigramCost = bigramCost

    self.possibleFills = possibleFills

---

def startState(self) -> Tuple[int,str]:

return (0,wordsegUtil.SENTENCE_BEGIN)

---

def isEnd(self, state: Tuple[int,str]) -> bool

return state[0] == len(self.query)

```
def succAndCost(self, state: Tuple[int,str]) -> Iterable[Tuple[str, Tuple[int,str],float]]:
succAndCost_list = []
start_idx = state[0]
for end_idx in range(start_idx+1, len(self.query)+1)
succ_set = self.possibleFills(self.query[start_idx:end_idx])
if len(succ_set) != 0
for succ in sorted(succ_set):  //->set을 반복문에 쓰기 위해 sort
cost = self.bigramCost(state[1],succ)
succAndCost_list.append((succ,(end_idx,succ),cost))
return succAndCost_list
================================================================
def segmentAndInsert(
  query: str,
  bigramCost: Callable[[str, str], float],
  possibleFills: Callable[[str], Set[str]],
) -> str:
if len(query) == 0:
return ""

ucs = util.UniformCostSearch(verbose=0)
ucs.solve(JointSegmentationInsertionProblem(query,bigramCost,possibleFills))
if ucs.actions is None:
return ""
return " ".join(ucs.actions)
```

3b÷ Exact-k Joint Reconstruction (k word + Restricted)

```
class ExactKJointReconstructionProblem(util.SearchProblem):
  def __init__(
    self,
    query: str,
    k: int,
    bigramCost: Callable[[str, str], float],
    possibleFills: Callable[[str], Set[str]],
  ):
    self.query = query
    self.k = k
    self.bigramCost = bigramCost
    self.possibleFills = possibleFills
```

```
---------------------------------------------------------------
def startState(self) -> Tuple[int,str,int]:
return (0,wordsegUtil.SENTENCE_BEGIN,0)
---------------------------------------------------------------
def isEnd(self, state: Tuple[int,str,int])->bool:
return state[0] == len(self.query) and state[2] = self.k
---------------------------------------------------------------
def succAndCost(self,state:Tuple[int,str,int])->Iterable[Tuple[str, Tuple[int, str, int], float]]:
if state[2] == self.k:
return []
succAndCost_list = []
start_idx = state[0]
for end_idx in range(start_idx+1, len(self.query)+1):
succ_set = self.possibleFills(self.query[strat_idx:end_idx])
if len(succ_set) != 0
for succ in sorted(succ_set):
cost = self.bigramCost(state[1],succ)
succAndCost_list.append((succ,(end_idx,succ,state[2]+1),cost))
return succAndCost_list

//빈 리스트 반환 -> 탐색하지 못하는 상태(막다른 길)을 보이는 것임
//              그래서 무한 루프에 빠지지 않는다
---------------------------------------------------------------
def segmentAndInsertExactK(
  query: str,
  k: int,
  bigramCost: Callable[[str, str], float],
  possibleFills: Callable[[str], Set[str]],
) -> str:
if len(query) == 0 or len(query) < k or k ==0:
return ""
ucs = utill.UniformCostSearch(verbose=0)
ucs.solve(ExactKJointReconstructionProblem(query,k,bigramCost,possibleFills))
if ucs.actions is None:
return ""
return " ".join(ucs.actions)
```

Problem 4 : *2-Waypoint Pathfinding*

---

basic 2-way SearchProblem - yield

```
class MultiWaypointPathFindingProblem(util.SearchProblem):
  def __init__(self, start: tuple, waypoints: list, goal: tuple,
moveCost: Callable[[tuple, tuple], float],
  possibleMoves: Callable[[tuple], Set[tuple]]):
self.start = start
    self.waypoints = list(waypoints)  # exactly 2 waypoints: [W1, W2]
    self.goal = goal
    self.moveCost = moveCost
    self.possibleMoves = possibleMoves
----------------------------------------------------------------
def startState(self):
return (self.start, False, False)
----------------------------------------------------------------
def isEnd(self, state) -> bool
position, visitedW1, visitiedW2 = state
return position == self.goal and visitedW1 and visitiedW2
----------------------------------------------------------------
def succAndCost(self,state):
position, visitedW1, visitedW2 = state
for direction, next_position in self.possibleMoves(position):
new_W1 = visitedW1 or (next_position == self.waypoints[0])
new_W2 = visitedW2 or (next_position == self.waypoints[1])
yield direction, (next_position, new_W1, new_w2), self.moveCost(position, direction)
----------------------------------------------------------------


def UCSMultiWaypointPathFindingSearch(start: tuple, waypoints: list, goal: tuple,
                                      moveCost: Callable[[tuple, tuple], float],
                                      possibleMoves: Callable[[tuple], Set[tuple]]) -> List[str]:
ucs = util.UniformCostSearch(verbose=0)
  ucs.solve(MultiWaypointPathFindingProblem(start, waypoints, goal, moveCost, possibleMoves))
  return ucs.actions
```

4a÷multiWaypointHeuristic

```
def multiWaypointHeuristic(waypoints: list, goal: tuple):
----------------------------------------------------------------
```

```
 def _multiWaypointHeuristic(state: tuple) -> float:
position, visitedW1, visitedW2 = state

def Manhattan_distance(pos1, pos2)
x1, y1 = pos1
x2, y2 = pos2
return abs(x1-x2)+abs(y1-y2)
h1 = Manhattan_distance(position, goal)
univisited_distance = [] //-> unvisited Waypoint와의 거리
if visitedW1 is False:
unvisited_distance.append(Manhattan_distance(position,waypoints[0]))
if visitedW2 is False:
unvisited_distance.append(Manhattan_distance(position,waypoints[1]))

h2 = min(nuvisited_distance) if len(nuvisited_distance) !=0 else 0
return max(h1, h2)
return _multiwaypointHeuristic

// h1 -> goal까지의 heuristic
// h2 -> waypoint까지의 heuristic
-------------------------------------------------------------
def AStarMultiWaypointPathFindingSearch(start: tuple, waypoints: list, goal: tuple,
                                        moveCost: Callable[[tuple, tuple], float],
                                        possibleMoves: Callable[[tuple], Set[tuple]]) -> List[str]:
heuristic_func = multiWaypointHeuristic(waypoints, goal)
 ucs = util.UniformCostSearch(verbose=0)
 problem = MultiWaypointPathFindingProblem(start, waypoints, goal, moveCost, possibleMoves)
 ucs.solve(problem, heuristic=heuristic_func) // 휴리스틱이 있을 때.
 return ucs.actions
```

← 인공지능 · 포스텍 목차로